

PLATEFORME BANCAIRE DISTRIBUÉE

INF462 – Systèmes Distribués

PLAN DE TRAVAIL COMPLET

10 Phases | 13 Microservices | 3 Frontends

TABLE DES MATIÈRES

TOC \h \o "1-3" \t "Heading1,1,Heading2,2,Heading3,3"HYPERLINK "" \l "_Toc231462978"TABLE DES MATIÈRES.....	PAGEREF _Toc231462978 \h 1
<i>INTRODUCTION & OBJECTIFS DU PROJET.....</i>	<i>5</i>
<i>Vision du Projet.....</i>	<i>5</i>
<i>Architecture Globale.....</i>	<i>5</i>
<i>Comment Utiliser ce Document.....</i>	<i>5</i>
<i>RÉCAPITULATIF DES TECHNOLOGIES.....</i>	<i>6</i>
<i>P0.1 – Créer les Repositories Git.....</i>	<i>8</i>
<i>P0.2 – Configurer l'Environnement Local.....</i>	<i>8</i>
<i>P0.3 – Définir les Standards du Projet.....</i>	<i>9</i>
<i>Git Flow.....</i>	<i>9</i>
<i>Ports des Services (à réserver dès maintenant).....</i>	<i>9</i>
<i>Format des Logs.....</i>	<i>9</i>
<i>DTOs Communs.....</i>	<i>9</i>
<i>P1.1 → P1.6 : Service Gestion Clients.....</i>	<i>11</i>
<i>P1.1 – Initialisation du Projet.....</i>	<i>11</i>
<i>P1.2 – Modélisation Base de Données.....</i>	<i>11</i>
<i>P1.3 – Endpoints REST.....</i>	<i>11</i>
<i>P1.4 – Logique Métier.....</i>	<i>12</i>
<i>P1.5 – Tests.....</i>	<i>12</i>
<i>P1.6 – Dockerisation.....</i>	<i>12</i>
<i>P1.7 → P1.11 : Service Gestion Comptes Financiers.....</i>	<i>12</i>
<i>P1.7 – Initialisation.....</i>	<i>12</i>
<i>P1.8 – Modélisation.....</i>	<i>12</i>
<i>P1.9 – Endpoints REST.....</i>	<i>13</i>
<i>P1.10 – Communication Inter-Services.....</i>	<i>13</i>
<i>P1.11 – Tests + Dockerisation.....</i>	<i>13</i>
<i>P1.12 → P1.17 : Service Transactions & Transferts.....</i>	<i>13</i>
<i>P1.12 – Initialisation.....</i>	<i>13</i>
<i>P1.13 – Modélisation MongoDB.....</i>	<i>13</i>
<i>P1.14 – Endpoints REST.....</i>	<i>14</i>
<i>P1.15 – Logique de Commission.....</i>	<i>14</i>
<i>P1.16 – Publication Kafka.....</i>	<i>14</i>
<i>P1.17 – Tests + Dockerisation.....</i>	<i>14</i>
<i>P2.1 → P2.5 : Service Prêts & Échéanciers (Python/FastAPI).....</i>	<i>15</i>
<i>P2.1 – Initialisation Python.....</i>	<i>15</i>
<i>P2.2 – Modèles de Données.....</i>	<i>15</i>

<u>P2.3 – Endpoints FastAPI.....</u>	<u>15</u>
<u>P2.4 – Intégration OCR.....</u>	<u>15</u>
<u>P2.5 – Tests + Dockerisation.....</u>	<u>16</u>
<u>P2.6 → P2.10 : Service IA OCR & Gestion Documentaire (Python).....</u>	<u>16</u>
<u>P2.6 – Initialisation.....</u>	<u>16</u>
<u>P2.7 – Pipeline OCR.....</u>	<u>16</u>
<u>P2.8 – Scoring Crédit.....</u>	<u>16</u>
<u>P2.9 – Stockage Documents.....</u>	<u>16</u>
<u>P2.10 – Tests + Dockerisation.....</u>	<u>17</u>
<u>P2.11 → P2.15 : Service Authentification (Node.js).....</u>	<u>17</u>
<u>P2.11 – Initialisation.....</u>	<u>17</u>
<u>P2.12 – Mécanismes d'Authentification.....</u>	<u>17</u>
<u>P2.13 – RBAC (Rôles et Permissions).....</u>	<u>17</u>
<u>P2.14 – Gestion des Sessions Redis.....</u>	<u>17</u>
<u>P2.15 – Tests + Dockerisation.....</u>	<u>17</u>
<u>P2.16 → P2.20 : Service Notifications (Node.js).....</u>	<u>18</u>
<u>P2.16 → P2.17 – Adapters Multi-Canaux.....</u>	<u>18</u>
<u>P2.18 → P2.19 – Consommateur Kafka + Templates.....</u>	<u>18</u>
<u>P2.20 – Tests + Dockerisation.....</u>	<u>18</u>
<u>P2.21 → P2.24 : Service Admin & Rapports (Node.js).....</u>	<u>18</u>
<u>P2.21 → P2.22 – Dashboard Admin.....</u>	<u>18</u>
<u>P2.23 – Génération de Rapports.....</u>	<u>18</u>
<u>P2.24 – Tests + Dockerisation.....</u>	<u>19</u>
<u>P2.25 → P2.30 : Service Audit & Traçabilité (Python).....</u>	<u>19</u>
<u>P2.25 → P2.27 – Consommateur & Modèle.....</u>	<u>19</u>
<u>P2.28 → P2.29 – API + Intégrité.....</u>	<u>19</u>
<u>P2.30 – Tests + Dockerisation.....</u>	<u>19</u>
<u>P3.1 – Déploiement Message Broker (Kafka).....</u>	<u>20</u>
<u>P3.2 – Communication Asynchrone.....</u>	<u>20</u>
<u>P3.3 – Communication Synchrone (REST).....</u>	<u>20</u>
<u>P3.4 – Spring Cloud Config.....</u>	<u>20</u>
<u>P3.5 – Consul (Service Discovery).....</u>	<u>20</u>
<u>P3.6 – Circuit Breakers.....</u>	<u>21</u>
<u>P4.1 → P4.7 : Application Web (ReactJS).....</u>	<u>22</u>
<u>P4.1 – Initialisation.....</u>	<u>22</u>
<u>P4.2 – Authentification.....</u>	<u>22</u>
<u>P4.3 → P4.5 – Dashboards.....</u>	<u>22</u>
<u>P4.6 → P4.7 – Gateway + Tests.....</u>	<u>22</u>
<u>P4.8 → P4.13 : Application Mobile (Flutter).....</u>	<u>22</u>

<i>P4.8 – Initialisation.....</i>	<i>23</i>
<i>P4.9 → P4.13 – Fonctionnalités.....</i>	<i>23</i>
<i>P5.1 – Kong (API Gateway).....</i>	<i>24</i>
<i>P5.2 – Istio (Service Mesh).....</i>	<i>24</i>
<i>P5.3 – Sécurisation Globale.....</i>	<i>24</i>
<i>P5.4 – Rate Limiting & Throttling.....</i>	<i>24</i>
<i>P6.1 → P6.4 : Centralized Logging (ELK Stack).....</i>	<i>25</i>
<i>P6.5 → P6.8 : Monitoring (Prometheus & Grafana).....</i>	<i>25</i>
<i>P6.9 → P6.11 : Distributed Tracing (Jaeger).....</i>	<i>25</i>
<i>P7.1 – Pipelines GitHub Actions.....</i>	<i>26</i>
<i>Workflow CI (ci.yml) – déclenché sur push/PR vers dev et main.....</i>	<i>26</i>
<i>Workflow CD (cd.yml) – déclenché sur push vers main.....</i>	<i>26</i>
<i>P7.2 – GitOps (ArgoCD).....</i>	<i>26</i>
<i>P7.3 → P7.4 – Versioning & Smoke Tests.....</i>	<i>26</i>
<i>P8.1 – Docker Compose (Développement & Démo).....</i>	<i>27</i>
<i>P8.2 – Kubernetes Local (k3s/minikube/k3d).....</i>	<i>27</i>
<i>P9.1 – Préparation du VPS.....</i>	<i>28</i>
<i>P9.2 – Kubernetes sur VPS (K3s).....</i>	<i>28</i>
<i>P9.3 – Déploiement des Services en Production.....</i>	<i>28</i>
<i>P9.4 → P9.5 – Observabilité + Backup.....</i>	<i>28</i>
<i>P10.1 – Tests E2E Complet.....</i>	<i>30</i>
<i>P10.2 – Tests de Charge.....</i>	<i>30</i>
<i>P10.3 – Tests de Chaos.....</i>	<i>30</i>
<i>P10.4 – Documentation Technique.....</i>	<i>30</i>
<i>P10.5 – Préparation de la Démonstration.....</i>	<i>30</i>
<i>PLANNING & ESTIMATION TEMPORELLE.....</i>	<i>32</i>
<i>Ordre de Priorité Recommandé.....</i>	<i>32</i>
<i>DESIGN PATTERNS À IMPLÉMENTER.....</i>	<i>33</i>
<i>Strategy Pattern – Calcul des Commissions.....</i>	<i>33</i>
<i>Observer Pattern (via Kafka) – Événements Métier.....</i>	<i>33</i>
<i>Circuit Breaker – Résilience.....</i>	<i>33</i>
<i>Saga Pattern – Transactions Distribuées.....</i>	<i>33</i>
<i>CQRS – Séparation Lecture/Écriture.....</i>	<i>33</i>
<i>CONSEILS POUR L'ÉQUIPE.....</i>	<i>34</i>
<i>Organisation du Travail en Équipe.....</i>	<i>34</i>
<i>Pièges à Éviter.....</i>	<i>34</i>
<i>Ressources Utiles.....</i>	<i>34</i>

INTRODUCTION & OBJECTIFS DU PROJET

Ce document constitue le plan de travail officiel du projet INF462 – Plateforme Bancaire Distribuée. Il détaille l'ensemble des tâches, sous-tâches et livrables attendus pour chacune des 10 phases du projet.

Vision du Projet

Concevoir et déployer une plateforme de services financiers multi-opérateurs, architecturée en microservices, multi-langages (Java, Python, Node.js), avec un frontend Web (React) et Mobile (Flutter), déployée en local puis sur VPS en production.

Architecture Globale

La plateforme comprend 13 composants principaux :

- 3 services Java/Spring Boot (Clients, Comptes, Transactions)
- 2 services Python/FastAPI (Prêts, IA-OCR, Audit)
- 3 services Node.js (Auth, Notifications, Admin)
- 2 frontends (React Web + Flutter Mobile)
- Infrastructure : Kong, Consul, Kafka, ELK, Prometheus, Jaeger

Comment Utiliser ce Document

Chaque phase est décomposée en tâches (Px.y) et sous-tâches à cocher (☐). Les cases à cocher permettent de suivre l'avancement en équipe. Les couleurs indiquent la phase.

- ☐ = tâche à réaliser
- Durée estimée en fin de chaque section
- Les dépendances sont mentionnées dans le texte

RÉCAPITULATIF DES TECHNOLOGIES

Couche	Technologie	Justification
Frontend Web	ReactJS 18+, Vite, TypeScript, TailwindCSS/MUI	Écosystème mature, SSR possible, large communauté
Frontend Mobile	Flutter 3.x (Dart)	Cross-platform iOS/Android avec une seule base de code
Services Core	Java 17, Spring Boot 3.x, Spring Cloud	Robustesse entreprise, Spring Cloud pour microservices
Services Spécialisés	Python 3.11+, FastAPI, SQLAlchemy	Performance async, bibliothèques IA/ML riches
Services Temps Réel	Node.js 20 LTS, Express, Socket.io	Event loop, idéal pour notifications temps réel
Base de données SQL	PostgreSQL 15+	ACID, transactions fiables pour données financières
Base de données NoSQL	MongoDB 7+	Flexibilité pour logs, transactions, documents
Cache / Sessions	Redis 7+	Ultra-rapide, blacklist JWT, sessions distribuées
Message Bus	Apache Kafka	Haut débit, durabilité, réplication des événements
API Gateway	Kong	Rate limiting, auth JWT, CORS, plugins extensibles
Service Mesh	Istio	mTLS, traffic management, observabilité avancée
Service Discovery	Consul	Health checks, DNS, configuration dynamique
Config Centralisée	Spring Cloud Config	Centralisation des configs Java avec hot-reload
Conteneurisation	Docker, Docker Compose	Isolation, reproductibilité des environnements
Orchestration	Kubernetes (K3s)	Auto-scaling, auto-healing, rolling updates
CI/CD	GitHub Actions	Automatisation lint/test/build/deploy
Logging	ELK Stack (Elasticsearch, Logstash, Kibana)	Centralisation et analyse des logs
Monitoring	Prometheus + Grafana + Alertmanager	Métriques temps réel, alertes
Tracing	Jaeger + OpenTelemetry	Traçage distribué des requêtes

<i>Sécurité</i>	<i>JWT, OAuth2, mTLS, Cert-Manager, HashiCorp Vault</i>	<i>Authentification multi-couches, secrets management</i>
<i>Tests</i>	<i>JUnit 5, pytest, Jest, Cypress, k6, Chaos Mesh</i>	<i>Couverture unitaire, intégration, charge, chaos</i>

PHASE 0 PRÉPARATION & ENVIRONNEMENT

3-5 jours

Objectif : Mettre en place tous les outils, la structure de projet et les standards avant tout développement. Cette phase est critique – un environnement mal configuré ralentit toute l'équipe.

P0.1 – Créer les Repositories Git

Créer 13 repositories distincts sur GitHub/GitLab avec la structure standard suivante :

- *src/ – code source*
- *tests/ – tests unitaires et d'intégration*
- *docs/ – documentation*
- *docker/ – Dockerfiles et compose files*

Liste des repositories à créer :

- ☐ *bank-platform-clients (Java/Spring Boot)*
- ☐ *bank-platform-accounts (Java/Spring Boot)*
- ☐ *bank-platform-transactions (Java/Spring Boot)*
- ☐ *bank-platform-loans (Python/FastAPI)*
- ☐ *bank-platform-ai-ocr (Python/TensorFlow)*
- ☐ *bank-platform-notifications (Node.js)*
- ☐ *bank-platform-auth (Node.js)*
- ☐ *bank-platform-admin-reports (Node.js)*
- ☐ *bank-platform-audit (Python)*
- ☐ *bank-platform-frontend-web (ReactJS)*
- ☐ *bank-platform-frontend-mobile (Flutter)*
- ☐ *bank-platform-infra (Docker, K8s, Terraform)*
- ☐ *bank-platform-docs (Documentation technique)*

Pour chaque repo, initialiser avec : .gitignore, README.md, LICENSE (MIT), structure de dossiers standard.

P0.2 – Configurer l'Environnement Local

Chaque membre de l'équipe doit installer les outils suivants et vérifier les versions :

- ☐ *JDK 17+ (java --version → doit afficher 17 ou +)*
- ☐ *Maven 3.9+ (mvn --version)*
- ☐ *Python 3.11+ avec pip et venv (python --version)*
- ☐ *Node.js 20 LTS + npm (node --version)*
- ☐ *Docker Desktop / Docker Engine (docker --version)*
- ☐ *Docker Compose v2 (docker compose version)*
- ☐ *kubectl (kubectl version)*

- *minikube* ou *k3d* pour tests Kubernetes locaux
- *Postman* ou *Insomnia* pour tester les APIs
- *pgAdmin* pour PostgreSQL, *MongoDB Compass* pour MongoDB
- *Git* configuré avec SSH key vers *GitHub*

*Livrable : créer un fichier **SETUP.md** dans *bank-platform-docs* listant les versions et commandes de vérification.*

P0.3 – Définir les Standards du Projet

Établir les conventions que TOUTE l'équipe respectera :

Git Flow

- *Branches : main (production), dev (intégration), feature/* (développement), hotfix/**
- *Convention de commit : feat:, fix:, docs:, test:, chore:*
- *Pull Request obligatoire pour merger dans dev et main*

Ports des Services (à réserver dès maintenant)

- *Client Service : 8081*
- *Account Service : 8082*
- *Transaction Service : 8083*
- *Auth Service : 8084*
- *Loan Service : 8085*
- *OCR Service : 8086*
- *Notification Service : 8087*
- *Admin Service : 8088*
- *Audit Service : 8089*
- *Kong (API Gateway) : 8000 (HTTP), 8443 (HTTPS), 8001 (Admin)*
- *Frontend Web : 3000*
- *PostgreSQL : 5432, MongoDB : 27017, Redis : 6379*
- *Kafka : 9092, ZooKeeper : 2181*
- *Consul : 8500, Prometheus : 9090, Grafana : 3001, Kibana : 5601, Jaeger : 16686*

Format des Logs

- *Format JSON structuré obligatoire pour tous les services*

Exemple : {"timestamp":"2025-01-01T10:00:00Z", "level":"INFO", "service":"client-service", "message":"Client created", "traceId":"abc123"}

DTOs Communs

- *Créer un module Maven partagé OU utiliser OpenAPI Generator*
- *Documenter toutes les APIs en OpenAPI/Swagger dès la création des endpoints*

Livrable : STANDARDS.md + docker-compose.dev.yml de base avec tous les ports mappés.

PHASE 1 MICROSERVICES CORE (Java/Spring Boot)

3 semaines

2-

Objectif : Développer les 3 services métier fondamentaux en Java/Spring Boot. Ces services forment le coeur financier de la plateforme.

P1.1 → P1.6 : Service Gestion Clients

Repository : bank-platform-clients | Port : 8081 | Base de données : PostgreSQL

Durée estimée : 3-5 jours

P1.1 – Initialisation du Projet

- Créer le projet Spring Boot 3.x via Spring Initializr (start.spring.io)
- Dépendances Maven à inclure : Spring Web, Spring Data JPA, Spring Validation, PostgreSQL Driver, Lombok, Spring Boot Actuator
- Configurer application.yml : datasource PostgreSQL, port 8081, logging JSON
- Vérifier le démarrage : GET http://localhost:8081/actuator/health doit retourner {status: UP}

P1.2 – Modélisation Base de Données

- Entité Client : id (UUID), nom, prénom, email (unique), téléphone (unique), dateNaissance, adresse, statut (ACTIVE/SUSPENDED/PENDING), dateCreation
- Entité KycDocument : id, clientId (FK), type (CNI/PASSEPORT/JUSTIF_DOMICILE), url, statutValidation (PENDING/VALIDATED/REJECTED), dateUpload
- Entité AffiliatedOperator : id, clientId (FK), operatorId, dateAffiliation – table de jointure Client ↔ Opérateur
- Créer les migrations Flyway ou Liquibase pour le versioning du schéma

P1.3 – Endpoints REST

- POST /api/clients – créer un client (valider unicité email+téléphone)
- GET /api/clients – liste paginée avec filtres (?nom=, ?email=, ?statut=)
- GET /api/clients/{id} – détail d'un client
- PUT /api/clients/{id} – mise à jour profil
- DELETE /api/clients/{id} – désactivation (soft delete, passer statut SUSPENDED)
- POST /api/clients/{id}/kyc – upload document KYC (MultipartFile → stockage MinIO/disque → retourner URL)
- GET /api/clients/{id}/kyc – liste des documents KYC

- `PUT /api/clients/{id}/operators` – associer à un opérateur

Important : utiliser des DTOs séparés (`ClientRequest`, `ClientResponse`) – ne JAMAIS exposer les entités JPA directement.

P1.4 – Logique Métier

- Valider unicité email ET téléphone avant création (retourner 409 Conflict si doublon)
- Règle plafond : un client ne peut pas avoir plus de 3 comptes actifs (vérification via appel au Account Service)
- Statut `PENDING` à la création, passage `ACTIVE` après validation KYC
- Implémenter un `@ControllerAdvice` global pour les exceptions (404, 400, 409, 500)

P1.5 – Tests

- Tests unitaires JUnit 5 + Mockito : tester la logique métier en isolation (`ClientService`, validation, transformation DTO)
- Tests d'intégration `@SpringBootTest` + `TestContainers` PostgreSQL : tester les repositories et les endpoints HTTP
- Couverture minimale requise : 70% des classes de service

P1.6 – Dockerisation

- Dockerfile multi-stage : stage 1 (`maven:3.9-openjdk-17` pour le build), stage 2 (`eclipse-temurin:17-jre` pour le runtime)
- `docker-compose.yml` de dev : `client-service` + PostgreSQL avec volumes persistants
- Tester : `docker compose up` → `GET http://localhost:8081/actuator/health`

P1.7 → P1.11 : Service Gestion Comptes Financiers

Repository : `bank-platform-accounts` | Port : 8082 | Base de données : PostgreSQL

Durée estimée : 3-4 jours

P1.7 – Initialisation

- Même stack Spring Boot 3.x que le `client-service`
- Dépendances supplémentaires : Spring WebFlux (pour `WebClient` non-bloquant)

P1.8 – Modélisation

- Entité `Account` : `id` (UUID), `clientId`, `type` (`EPARGNE/COURANT/MOBILE_MONEY`), `solde` (`BigDecimal` – JAMAIS `float/double` pour l'argent), `devise` (`XAF/EUR/USD`), `statut`, `dateCreation`, `dateClôture`

- Entité `AccountType` : règles par opérateur : `plafondDepot`, `plafondRetrait`, `plafondSolde`, `fraisGestion`
- Entité `BalanceHistory` : snapshot quotidien du solde pour les rapports

P1.9 – Endpoints REST

- `POST /api/accounts` – créer un compte (vérifier que le client existe via REST call vers `client-service`)
- `GET /api/accounts/{id}` – détail compte + solde
- `GET /api/accounts/client/{clientId}` – tous les comptes d'un client
- `PUT /api/accounts/{id}/suspend` – suspendre un compte
- `PUT /api/accounts/{id}/close` – clôturer un compte
- `PUT /api/accounts/{id}/balance` – mettre à jour le solde (appelé par `transaction-service`)
- `GET /api/accounts/{id}/history` – historique des soldes (`BalanceHistory`)

P1.10 – Communication Inter-Services

- Configurer `WebClient` (Spring `WebFlux`) pour appeler `client-service` lors de la création d'un compte
- Gérer les cas d'erreur : si `client-service` est down → retourner 503 avec message clair
- Ajouter `Resilience4j` : `@CircuitBreaker(name = 'clientService', fallbackMethod = ...)`

P1.11 – Tests + Dockerisation

- Tests unitaires sur `AccountService` (logique plafonds, validation)
- Tests intégration avec `TestContainers PostgreSQL`
- `Dockerfile multi-stage` + `docker-compose.yml`

P1.12 → P1.17 : Service Transactions & Transferts

Repository : `bank-platform-transactions` | Port : 8083 | Base de données : MongoDB

Durée estimée : 4-6 jours – le plus complexe des services core

P1.12 – Initialisation

- Spring Boot 3.x avec Spring Data MongoDB (pas JPA – MongoDB pour la flexibilité des documents de transaction)
- Dépendances : Spring Web, Spring Data MongoDB, Spring Kafka, Lombok, Validation

P1.13 – Modélisation MongoDB

- Document `Transaction` : `id`, `accountId`, `type` (`DEPOSIT`/`WITHDRAW`/`TRANSFER_IN`/`TRANSFER_OUT`), `montant`, `devise`, `statut`

(PENDING/COMPLETED/FAILED/REVERSED), commission, commissionRate, operatorId, description, metadata (Map flexible), timestamp

- Document Transfer : id, sourceAccountId, destinationAccountId, montant, frais, statut, type (INTERNAL/EXTERNAL), timestamp
- Créer des index MongoDB sur : accountId, timestamp, statut (pour les requêtes de filtrage)

P1.14 – Endpoints REST

- POST /api/transactions/deposit – déposer de l'argent
- POST /api/transactions/withdraw – retirer de l'argent
- POST /api/transfers/internal – transfert entre comptes du même opérateur
- POST /api/transfers/external – transfert inter-opérateurs (Wave → Orange Money)
- GET /api/transactions – historique filtrable (?accountId=, ?type=, ?from=, ?to=, ?statut=) avec pagination
- GET /api/transactions/{id} – détail d'une transaction

Règle critique : Toujours vérifier le solde ET les plafonds avant d'exécuter une transaction. Utiliser des transactions MongoDB multi-documents si nécessaire.

P1.15 – Logique de Commission

- Créer une table/configuration de taux de commission par opérateur (ex: Wave=1%, Orange Money=2%, MTN=1.5%)
- Calculer la commission sur chaque transaction et la stocker dans le document Transaction
- Stratégie Pattern recommandé : interface CommissionStrategy avec implémentations par opérateur

P1.16 – Publication Kafka

- Configurer Spring Kafka (KafkaTemplate)
- Publier TransactionCreatedEvent sur le topic 'transactions.created' après chaque transaction réussie
- Publier aussi sur 'audit.event' pour la traçabilité
- Format d'événement JSON : {eventId, eventType, timestamp, payload: {transaction}}

P1.17 – Tests + Dockerisation

- Tests unitaires sur la logique de commission et validation
- Tests intégration avec TestContainers MongoDB
- Dockerfile multi-stage + docker-compose avec MongoDB

PHASE 2 MICROSERVICES SPÉCIALISÉS

2-3 semaines

P2.1 → P2.5 : Service Prêts & Échéanciers (Python/FastAPI)

Repository : bank-platform-loans | Port : 8085 | Base de données :

PostgreSQL (SQLAlchemy)

Durée estimée : 4-5 jours

P2.1 – Initialisation Python

- Créer le projet : `python -m venv venv + pip install fastapi uvicorn sqlalchemy pydantic httpx pytest`
- Structure : `app/models/`, `app/routers/`, `app/services/`, `app/schemas/`
- Configuration Alembic pour les migrations de base de données

P2.2 – Modèles de Données

- Modèle `LoanRequest` : `id`, `clientId`, `montant`, `dureeEnMois`, `objetDuPret`, `statut` (PENDING/APPROVED/REJECTED/DISBURSED), `dateCreation`
- Modèle `CreditDossier` : `id`, `loanRequestId`, `score` (0-100), `decision`, `motifRefus`, `documentsJustificatifs`
- Modèle `RepaymentSchedule` : `id`, `loanId`, `totalEcheances`, liste d'Echeance (`date`, `montantTotal`, `capitalDu`, `interets`, `capitalRestant`)
- Modèle `Repayment` : `id`, `loanId`, `montantPaye`, `datePaiement`, `echeanceId`, `statut`

P2.3 – Endpoints FastAPI

- `POST /api/loans/requests` – soumettre une demande de prêt
- `GET /api/loans/requests` – lister les demandes (filtres par statut, `clientId`)
- `POST /api/loans/simulate` – simuler un prêt (calcul mensualité, coût total, TAEG) sans créer de dossier
- `PUT /api/loans/{id}/validate` – valider ou rejeter un dossier (rôle ADMIN/OPERATOR)
- `GET /api/loans/{id}/schedule` – obtenir l'échéancier de remboursement
- `POST /api/loans/{id}/repay` – enregistrer un remboursement

Formule mensualité : $M = C \times [t(1+t)^n] / [(1+t)^n - 1]$ où t = taux mensuel, n = nombre de mois, C = capital

P2.4 – Intégration OCR

- Lors du dépôt de documents (fiche de paie, CNI), appeler le service OCR via `httpx`
- Extraire les données (revenus, identité) et les stocker dans `CreditDossier`
- Gestion asynchrone : ne pas bloquer si l'OCR est lent

P2.5 – Tests + Dockerisation

- Tests pytest : tester le calcul de mensualité, les validations, les endpoints
- Dockerfile : python:3.11-slim, multi-stage si nécessaire

P2.6 → P2.10 : Service IA OCR & Gestion Documentaire (Python)

Repository : bank-platform-ai-ocr | Port : 8086 | Stockage : MinIO

Durée estimée : 5-7 jours (le plus complexe – dépend de la qualité des images)

P2.6 – Initialisation

- `pip install fastapi pyesseract easyocr opencv-python pdf2image scikit-learn boto3` (pour MinIO)
- Installer Tesseract-OCR sur le système (`apt install tesseract-ocr`)
- Configurer les langues Tesseract : français (fra) et anglais (eng)

P2.7 – Pipeline OCR

- `POST /api/ocr/extract` : reçoit un fichier image ou PDF
- Étape 1 – Conversion : si PDF → convertir en image avec pdf2image (poppler)
- Étape 2 – Prétraitement : redressement (deskewing), binarisation, réduction du bruit avec OpenCV
- Étape 3 – OCR : extraire le texte avec Tesseract ou EasyOCR
- Étape 4 – Parsing : identifier les champs (nom, prénom, numéro, dates, montants) avec regex ou NER
- Retourner un JSON structuré : {nom, prenom, numeroId, dateNaissance, revenus, ...}

P2.8 – Scoring Crédit

- Modèle simple de scoring : prendre en compte (revenus, ancienneté emploi, historique crédit, montant demandé/revenus)
- Implémenter avec scikit-learn (RandomForest ou LogisticRegression) ou règles métier simples
- Retourner un score 0-100 et une recommandation (APPROVE/REJECT/MANUAL_REVIEW)

P2.9 – Stockage Documents

- Configurer MinIO (S3-compatible) pour le stockage des fichiers
- Stocker les métadonnées dans MongoDB (nom fichier, URL, hash, statut OCR, date)

- Générer des URLs signées (pre-signed URLs) pour l'accès sécurisé aux documents

P2.10 – Tests + Dockerisation

- Tests avec des images de test (CNI fictives, fiches de paie simulées)
- Dockerfile avec toutes les dépendances systèmes (tesseract, poppler)

P2.11 → P2.15 : Service Authentication (Node.js)

Repository : bank-platform-auth | Port : 8084 | Base de données : PostgreSQL + Redis

Durée estimée : 4-5 jours

P2.11 – Initialisation

- `npm init + npm install express passport passport-jwt passport-google-oauth20 jsonwebtoken bcrypt redis speakeasy qrcode joi`
- `npm install --save-dev jest supertest nodemon`

P2.12 – Mécanismes d'Authentication

- Login email/password : vérification bcrypt, génération JWT (access token 15min + refresh token 7 jours)
- OAuth2 Google : Passport.js strategy Google OAuth2 – permettre login avec compte Google
- 2FA TOTP : générer un secret avec speakeasy, générer un QR code, vérifier le code 6 chiffres
- Biométrie simulée : endpoint POST /api/auth/biometric qui accepte un token de fingerprint et valide

P2.13 – RBAC (Rôles et Permissions)

- Définir les rôles : CLIENT, OPERATOR, ADMIN, SUPER_ADMIN
- Middleware authorize(...roles) qui lit le JWT et vérifie le rôle
- Table user_permissions en base pour les permissions granulaires

P2.14 – Gestion des Sessions Redis

- À la connexion : stocker la session dans Redis avec TTL (ex: 24h)
- À la déconnexion : ajouter le JWT à la blacklist Redis
- Middleware validateToken : vérifier que le token n'est pas blacklisté avant chaque requête protégée

P2.15 – Tests + Dockerisation

- Tests Jest + Supertest pour tous les endpoints d'auth
- Docker avec Node 20 alpine + Redis dans le compose

P2.16 → P2.20 : Service Notifications (Node.js)

Repository : bank-platform-notifications | Port : 8087

Durée estimée : 3-4 jours

P2.16 → P2.17 – Adapters Multi-Canaux

- EmailAdapter : utiliser Nodemailer (SMTP local pour dev, SendGrid pour prod)
- SMSAdapter : Twilio API (ou simulateur console.log en dev)
- PushAdapter : Firebase Admin SDK (firebase-admin) pour les notifications mobiles
- InAppAdapter : Socket.io rooms – chaque utilisateur rejoint une room par son userId

Pattern Adapter : chaque adapter implémente la même interface
{send(notification): Promise<void>}

P2.18 → P2.19 – Consommateur Kafka + Templates

- Configurer KafkaJS (consumer group notifications-group)
- Écouter le topic 'notifications.send' et router vers le bon adapter
- Créer des templates Handlebars par type d'événement et par opérateur

Exemple template : 'Bonjour {{name}}, votre transfert de {{amount}}
{{currency}} est confirmé. Référence : {{transactionId}}'

P2.20 – Tests + Dockerisation

- Tests Jest pour les adapters (mock Nodemailer, Twilio)
- Tests d'intégration du consumer Kafka

P2.21 → P2.24 : Service Admin & Rapports (Node.js)

Repository : bank-platform-admin-reports | Port : 8088

Durée estimée : 4-5 jours

P2.21 → P2.22 – Dashboard Admin

- CRUD Opérateurs : créer/modifier/désactiver des opérateurs financiers
- CRUD Administrateurs : gestion des comptes admin
- Paramétrage des règles métier : taux de commission, plafonds par opérateur (stockés en base, lus dynamiquement par les autres services)

P2.23 – Génération de Rapports

- Rapport transactions : GET /api/reports/transactions?
from=&to=&format=pdf|xlsx
- Rapport prêts : GET /api/reports/loans

- Implémenter PDF avec puppeteer (HTML → PDF) ou pdfkit
- Implémenter Excel avec exceljs

P2.24 – Tests + Dockerisation

- Tests sur la génération de rapports
- Dockerisation

P2.25 → P2.30 : Service Audit & Traçabilité (Python)

Repository : bank-platform-audit | Port : 8089 | Base de données : MongoDB

Durée estimée : 3-4 jours

P2.25 → P2.27 – Consommateur & Modèle

- Configurer un consumer Kafka Python (kafka-python ou aiokafka) écoutant TOUS les topics
- Modèle AuditLog : timestamp, userId, action (CREATE/UPDATE/DELETE/LOGIN/LOGOUT), resource, resourceId, serviceName, ipAddress, result (SUCCESS/FAILURE), changesJson (diff avant/après)
- Persister dans MongoDB – collection write-once (pas d'endpoint DELETE)

P2.28 → P2.29 – API + Intégrité

- GET /api/audit avec filtres et pagination (?from=, ?to=, ?service=, ?userId=, ?action=)
- Hash chaîné SHA-256 : chaque log contient le hash du log précédent + son contenu pour garantir l'intégrité
- Endpoint GET /api/audit/verify : vérifier l'intégrité de la chaîne de hashes

P2.30 – Tests + Dockerisation

- Tests pytest sur le consumer et l'API
- Dockerisation

PHASE 3 INFRASTRUCTURE DE COMMUNICATION

1 semaine

P3.1 – Déploiement Message Broker (Kafka)

- Déployer Apache Kafka avec ZooKeeper via Docker Compose
- Créer les topics : `transactions.created`, `loans.requested`, `notifications.send`, `audit.event`, `accounts.updated`
- Configurer la rétention : 7 jours pour les transactions, 30 jours pour l'audit
- Tester avec `kafka-console-producer` et `kafka-console-consumer`

P3.2 – Communication Asynchrone

- Transaction Service : publier `TransactionCreatedEvent` après chaque transaction
- Loan Service : publier `LoanRequestedEvent` à chaque nouvelle demande
- Account Service : publier `AccountUpdatedEvent` à chaque modification de solde
- Notification Service : consommer `notifications.send` et envoyer
- Audit Service : consommer TOUS les topics et persister

P3.3 – Communication Synchrone (REST)

- Client Service → Account Service : vérifier l'existence d'un client
- Account Service → Transaction Service : récupérer l'historique
- Loan Service → OCR Service : envoyer des documents pour extraction
- Utiliser `WebClient` (Java), `httpx` (Python), `axios` (Node) avec timeouts configurés (5s)

P3.4 – Spring Cloud Config

- Finaliser le `cloud-conf/` existant avec les configs de TOUS les services Java
- Configurer le Config Server pour servir les fichiers par profil (dev, prod)
- Les services Java se connectent au Config Server au démarrage

P3.5 – Consul (Service Discovery)

- Déployer Consul via Docker (port 8500)
- Chaque service Java s'enregistre : `spring-cloud-starter-consul-discovery`
- Health checks configurés sur `/actuator/health` pour les services Java
- Health checks HTTP custom pour Python et Node

P3.6 – Circuit Breakers

- *Java : Resilience4j – @CircuitBreaker, @Retry, @RateLimiter sur tous les appels HTTP sortants*
- *Python : pybreaker ou logique custom*
- *Node.js : opossum library*
- *Configurer les fallbacks : retourner des réponses dégradées plutôt que des erreurs*

PHASE 4 FRONTEND & CANAUX D'ACCÈS

2-3 semaines

P4.1 → P4.7 : Application Web (ReactJS)

Repository : *bank-platform-frontend-web* | Port : 3000

Durée estimée : 2-3 semaines

P4.1 – Initialisation

- `npm create vite@latest web-app -- --template react-ts`
- `npm install tailwindcss @headlessui/react react-router-dom axios zustand`
- `npm install recharts @tanstack/react-query`
- Configurer ESLint + Prettier + Husky (pre-commit hooks)

P4.2 – Authentification

- Page Login : formulaire email/password, gestion erreurs, redirect
- Gestion JWT : stocker en httpOnly cookie (plus sécurisé) ou localStorage
- Page 2FA : saisie du code TOTP à 6 chiffres
- Refresh token automatique : interceptor Axios qui renouvelle le token expiré
- Route guard : rediriger vers /login si non authentifié

P4.3 → P4.5 – Dashboards

- Dashboard Client : liste des comptes avec soldes, historique des transactions (table triable + filtres), formulaire de transfert avec confirmation en 2 étapes, formulaire demande de prêt + suivi
- Dashboard Opérateur : liste des transactions en attente de validation avec boutons Valider/Rejeter, gestion des clients (liste + recherche), validation des dossiers de prêt avec scoring affiché
- Dashboard Admin : KPIs globaux (nb clients, volume transactions, total prêts), graphiques temps réel (recharts), paramétrage des règles métier, gestion des utilisateurs

P4.6 → P4.7 – Gateway + Tests

- Toutes les requêtes API passent par Kong (`http://localhost:8000/api/...`)
- Configurer les variables d'environnement Vite (`VITE_API_BASE_URL`)
- Tests E2E Cypress : scénario login → dépôt → vérification solde
- Dockerisation : Nginx pour servir le build statique

P4.8 → P4.13 : Application Mobile (Flutter)

Repository : *bank-platform-frontend-mobile*

Durée estimée : 2-3 semaines

P4.8 – Initialisation

- *flutter create mobile_app --org com.bankplatform*
- *Architecture Clean : dossiers data/ (repos, datasources), domain/ (entities, usecases), presentation/ (screens, widgets, blocs)*
- *State management : flutter_bloc ou riverpod*
- *Dépendances : dio, flutter_secure_storage, local_auth, image_picker, firebase_messaging*

P4.9 → P4.13 – Fonctionnalités

- *Auth : écran de login, stockage token sécurisé (flutter_secure_storage), 2FA, biométrique (local_auth pour empreinte/Face ID)*
- *Fonctionnalités client : mêmes écrans que le web, adaptés mobile (comptes, transactions, transfert, prêts)*
- *Scan de documents : image_picker + caméra → upload multipart vers OCR Service*
- *Push notifications : firebase_messaging, notification in-app*
- *Tests unitaires/widget + flutter build apk pour Android, flutter build ios pour iOS*

PHASE 5 API GATEWAY & SÉCURITÉ

1 semaine

P5.1 – Kong (API Gateway)

- Déployer Kong via Docker (port 8000 HTTP, 8443 HTTPS, 8001 Admin API)
- Configurer les routes : `/api/clients` → `client-service:8081`, `/api/accounts` → `account-service:8082`, etc.
- Plugin Rate Limiting : 100 req/min par IP, 1000 req/min par service
- Plugin JWT Validation : vérifier le token avant de passer la requête au service
- Plugin CORS : configurer les origines autorisées (frontend React + Flutter)
- Plugin Request Transform : ajouter des headers (`X-Service-Name`, `X-Trace-Id`)

P5.2 – Istio (Service Mesh)

- Déployer Istio dans le cluster Kubernetes local
- Activer mTLS (mutual TLS) entre tous les services – communication interne chiffrée
- Configurer VirtualService et DestinationRule pour le trafic management
- Configurer Istio Gateway comme alternative à Kong ou complément

P5.3 – Sécurisation Globale

- HTTPS : certificats auto-signés en local avec mkcert, Let's Encrypt en production
- Secrets : utiliser Kubernetes Secrets ou HashiCorp Vault – JAMAIS de credentials en clair dans le code
- Sanitization Java : Jakarta Validation (`@NotNull`, `@Email`, `@Size`, `@Pattern`)
- Sanitization Python : Pydantic validators (type-safe automatiquement)
- Sanitization Node : Joi schema validation pour toutes les requêtes entrantes
- Protection SQL Injection : utiliser les ORM (JPA, SQLAlchemy) – ne JAMAIS concatener des requêtes

P5.4 – Rate Limiting & Throttling

- Kong pour le rate limiting global (par IP, par JWT, par service)
- Resilience4j `@RateLimiter` pour le rate limiting par service Java
- `express-rate-limit` pour les services Node.js

PHASE 6 OBSERVABILITÉ, LOGGING & MONITORING
semaine

1

P6.1 → P6.4 : Centralized Logging (ELK Stack)

- Déployer Elasticsearch 8.x (single-node pour dev, port 9200)
- Déployer Logstash avec pipeline de parsing des logs JSON
- Déployer Kibana (port 5601) avec dashboards préconfigurés : logs par service, erreurs, temps de réponse
- Configurer Filebeat sur chaque container pour collecter et envoyer les logs

Note : Elasticsearch est gourmand en RAM. En local : `-e ES_JAVA_OPTS='-Xms512m -Xmx512m'`

P6.5 → P6.8 : Monitoring (Prometheus & Grafana)

- Déployer Prometheus (port 9090) avec scrape config pour tous les services
- Java : ajouter micrometer-registry-prometheus + endpoint `/actuator/prometheus`
- Python : `pip install prometheus-client` + exposer `/metrics`
- Node.js : `npm install prom-client` + exposer `/metrics`
- Déployer Grafana (port 3001) avec dashboards : health des services, latence (p50/p95/p99), taux d'erreur, CPU/Mémoire
- Configurer Alertmanager : alertes Slack/email si service down > 1 minute

P6.9 → P6.11 : Distributed Tracing (Jaeger)

- Déployer Jaeger all-in-one pour le dev (port 16686 UI, port 14268 API)
- Java : ajouter OpenTelemetry Java Agent (javaagent) + configurer OTLP exporter
- Python : `pip install opentelemetry-sdk opentelemetry-instrumentation-fastapi`
- Node.js : `npm install @opentelemetry/sdk-node`
- Propagation du traceId dans tous les headers HTTP (X-Trace-Id)
- Vérifier dans Jaeger UI qu'une requête utilisateur traverse tous les services avec le même traceId

PHASE 7 CI/CD & AUTOMATISATION

1 semaine

P7.1 – Pipelines GitHub Actions

Créer dans chaque repository les fichiers `.github/workflows/` :

Workflow CI (ci.yml) – déclenché sur push/PR vers dev et main

- ❑ Job 1 - Lint : `eslint` (Node), `flake8` (Python), `checkstyle` (Java)
- ❑ Job 2 - Tests : `jest` (Node), `pytest` (Python), `mvn test` (Java)
- ❑ Job 3 - Build : `docker build` (toutes les langues)
- ❑ Job 4 - Scan sécurité : `Trivy` sur l'image Docker construite
- ❑ Job 5 - Push : pousser l'image vers Docker Hub ou GitHub Container Registry

Workflow CD (cd.yml) – déclenché sur push vers main

- ❑ Déploiement automatique sur Kubernetes via `kubectl apply` ou `helm upgrade`
- ❑ Smoke tests post-déploiement

P7.2 – GitOps (ArgoCD)

- ❑ Déployer ArgoCD dans le cluster Kubernetes
- ❑ Créer `bank-platform-infra` avec tous les manifests K8s
- ❑ Configurer ArgoCD pour surveiller ce repo et synchroniser automatiquement

P7.3 → P7.4 – Versioning & Smoke Tests

- ❑ Semantic Versioning : taguer les releases `v1.0.0`, `v1.1.0` (`feat` = `minor`, `fix` = `patch`, `breaking` = `major`)
- ❑ Images Docker taguées avec la version : `myapp:v1.0.0` ET `myapp:latest`
- ❑ Script `smoke-test.sh` : après déploiement, vérifier que `/actuator/health` répond 200 pour tous les services

PHASE 8 DÉPLOIEMENT LOCAL 1 semaine**P8.1 – Docker Compose (Développement & Démo)**

Créer un `docker-compose.yml` racine qui orchestre l'ensemble de la plateforme :

- ❑ Tous les microservices (build context pointant vers chaque dossier)
- ❑ PostgreSQL 15 avec volumes persistants (3 instances : clients, accounts, loans)
- ❑ MongoDB 7 avec volume persistant (transactions, audit, OCR)
- ❑ Redis 7
- ❑ Kafka + ZooKeeper
- ❑ Kong avec base de données (PostgreSQL dédié)
- ❑ Consul
- ❑ MinIO (stockage documents)
- ❑ ELK Stack (Elasticsearch + Logstash + Kibana + Filebeat)
- ❑ Prometheus + Grafana + Alertmanager
- ❑ Jaeger
- ❑ Frontend Web (Nginx + build React)

- ❑ Script de démarrage : `make up` ou `./start-local.sh` avec vérification de la santé de chaque service
- ❑ Tests de charge locaux avec `k6` ou `Artillery` pour valider la résilience

P8.2 – Kubernetes Local (k3s/minikube/k3d)

Écrire les manifests Kubernetes pour chaque service :

- ❑ Deployment : image, replicas=2, liveness probe (`/actuator/health`), readiness probe, resource limits (CPU/RAM)
- ❑ Service : ClusterIP pour les services internes
- ❑ ConfigMap : variables d'environnement non-sensibles
- ❑ Secret : credentials (base de données, JWT secret, API keys)
- ❑ PersistentVolumeClaim : pour PostgreSQL, MongoDB, MinIO, Elasticsearch
- ❑ Déployer NGINX Ingress Controller
- ❑ Déployer Istio + mTLS
- ❑ Déployer la stack monitoring via Helm charts (kube-prometheus-stack, ECK)
- ❑ Test failover : `kubectl delete pod <nom>` → vérifier le redémarrage automatique
- ❑ Test scalabilité : `kubectl scale deployment client-service -- replicas=3` → vérifier le load balancing

PHASE 9 DÉPLOIEMENT VPS PRODUCTION

1-2 semaines

P9.1 – Préparation du VPS

- Choisir un provider VPS : Hetzner, OVH, DigitalOcean – minimum 4 vCPU / 8 Go RAM / 100 Go SSD
- Provisionner Ubuntu 22.04 LTS
- Sécurisation : SSH key-only (désactiver authentication par mot de passe), désactiver root login
- Firewall UFW : ouvrir seulement les ports 22 (SSH), 80 (HTTP), 443 (HTTPS), 6443 (K8s API)
- Installer Fail2ban pour protection contre les attaques brute force
- Configurer unattended-upgrades pour les patches de sécurité automatiques

P9.2 – Kubernetes sur VPS (K3s)

- Installer K3s : `curl -sL https://get.k3s.io | sh -`
- Configurer kubectl local pour pointer sur le cluster distant (KUBECONFIG)
- Installer Helm 3 sur le cluster
- Configurer le Container Registry : Docker Hub recommandé pour démarrer

P9.3 – Déploiement des Services en Production

- Adapter les manifests K8s : ressources CPU/RAM réalistes, Pod Disruption Budgets, NetworkPolicies
- Déployer PostgreSQL en StatefulSet + PVC
- Déployer MongoDB en StatefulSet + PVC
- Déployer Redis en StatefulSet
- Déployer Kafka
- Déployer tous les microservices (images depuis CI/CD)
- Déployer Kong exposé sur les ports 80/443
- Configurer les DNS : A record vers l'IP du VPS
- SSL automatique : Cert-Manager + Let's Encrypt (certificats renouvelés automatiquement)

P9.4 → P9.5 – Observabilité + Backup

- Déployer ELK (ou Grafana Loki comme alternative légère)
- Prometheus + Grafana exposés via Ingress avec authentication basique
- Jaeger
- Alertes email/Slack pour la production
- Backups automatiques des PVCs avec Velero ou scripts cron vers S3
- Rotation des logs pour ne pas remplir le disque

- *Scan de sécurité Trivy régulier des images*
- *Documenter le runbook : procédures d'incident, procédure de rollback*

PHASE 10 TESTS FINAUX & DOCUMENTATION

1 semaine

P10.1 – Tests E2E Complet

Scénario à automatiser de bout en bout :

1. Inscription d'un nouveau client → validation KYC
2. Création d'un compte → dépôt initial
3. Transfert vers un autre compte → vérification soldes des deux comptes
4. Demande de prêt avec upload de documents → scoring OCR automatique
5. Validation du prêt par un admin
6. Enregistrement d'un remboursement → mise à jour de l'échéancier

P10.2 – Tests de Charge

- Outils : k6 (recommandé), JMeter ou Artillery
- Scénario : 100 utilisateurs simultanés effectuant des transactions
- Métriques à mesurer : latence moyenne, p95, p99, throughput (req/sec), taux d'erreur
- Objectifs cibles : p95 < 500ms pour les transactions, taux d'erreur < 1%

P10.3 – Tests de Chaos

- Outil : Chaos Mesh (K8s) ou tests manuels
- Scénario 1 : tuer aléatoirement des pods → vérifier la récupération automatique
- Scénario 2 : introduire une latence réseau de 500ms → vérifier les circuit breakers
- Scénario 3 : remplir le disque d'un service → vérifier les alertes

P10.4 – Documentation Technique

- Architecture C4 Model (Context, Container, Component, Code) – outil : draw.io ou structurizr
- INSTALL.md : guide d'installation locale pas-à-pas
- DEPLOY.md : guide de déploiement VPS
- API Reference : agréger les Swagger/OpenAPI de tous les services via Kong Developer Portal
- DESIGN_PATTERNS.md : documenter les patterns utilisés (Strategy pour commissions, Observer pour événements, Circuit Breaker, Saga pour transactions distribuées)

P10.5 – Préparation de la Démonstration

- *Scénario de démo fluide de 5-10 minutes (répéter plusieurs fois en équipe)*
- *Slides de présentation : architecture générale, choix technologiques justifiés, difficultés rencontrées et solutions*
- *Démo live du dashboard Grafana avec la plateforme sous charge*
- *Préparer un plan B si la démo live échoue (captures d'écran, vidéo pré-enregistrée)*

PLANNING & ESTIMATION TEMPORELLE

Phase	Description	Durée Estimée
Phase 0	Préparation & Environnement	3-5 jours
Phase 1	Services Core Java (3 services)	2-3 semaines
Phase 2	Services Spécialisés (5 services)	2-3 semaines
Phase 3	Infrastructure Communication (Kafka, Consul)	1 semaine
Phase 4	Frontend Web (React) + Mobile (Flutter)	2-3 semaines
Phase 5	API Gateway Kong + Sécurité Istio	1 semaine
Phase 6	Observabilité ELK + Prometheus + Jaeger	1 semaine
Phase 7	CI/CD GitHub Actions + ArgoCD	1 semaine
Phase 8	Déploiement Local Docker Compose + K8s	1 semaine
Phase 9	Déploiement VPS Production K3s	1-2 semaines
Phase 10	Tests Finaux + Documentation + Démo	1 semaine
TOTAL	À 2-3 personnes en parallèle	13-17 semaines

Conseil : Les phases 0-1-2 peuvent être développées en parallèle par différents membres de l'équipe. Déployer en local (Phase 8 partielle) dès qu'un service est terminé pour une boucle de feedback rapide.

Ordre de Priorité Recommandé

En cas de contrainte de temps, voici l'ordre de priorité :

7. Phase 0 + Phase 1 (Services Core) – le minimum viable
8. Phase 2 – Auth d'abord, puis Notifications
9. Phase 4 – Frontend Web (sans le mobile si manque de temps)
10. Phase 3 + Phase 5 – Communication et sécurité
11. Phase 8 – Déploiement local
12. Phase 6 + Phase 7 – Observabilité et CI/CD
13. Phase 9 + Phase 10 – Production et tests finaux

DESIGN PATTERNS À IMPLÉMENTER

Documenter ces patterns dans `DESIGN_PATTERNS.md` pour la Phase 10.

Strategy Pattern – Calcul des Commissions

Contexte : chaque opérateur financier a des règles de commission différentes.

- Interface `CommissionStrategy` avec méthode `calculate(montant): BigDecimal`
- Implémentations : `WaveCommissionStrategy` (1%), `OrangeMoneyStrategy` (2%), `MTNStrategy` (1.5%)
- Factory qui retourne la bonne stratégie selon l'opérateur

Observer Pattern (via Kafka) – Événements Métier

Contexte : propager les changements d'état à travers les services.

- Transaction Service publie `TransactionCreatedEvent` sur Kafka
- Notification Service et Audit Service s'y abonnent indépendamment
- Découplage total : Transaction Service ne sait pas qui écoute

Circuit Breaker – Résilience

Contexte : éviter les cascades de pannes entre services.

- `Resilience4j` sur les appels entre services Java
- État `CLOSED` (normal) → `OPEN` (trop d'erreurs) → `HALF-OPEN` (test)
- Fallback : retourner une réponse dégradée plutôt qu'une erreur 500

Saga Pattern – Transactions Distribuées

Contexte : un transfert inter-opérateurs implique plusieurs services.

- Saga Orchestrée : un orchestrateur coordonne les étapes
- Étapes : débit source → crédit destination → notification → audit
- Compensation : si une étape échoue, annuler les étapes précédentes

CQRS – Séparation Lecture/Écriture

Optionnel mais recommandé pour le service de transactions à fort volume.

- Command : écrire une nouvelle transaction (POST)
- Query : lire l'historique des transactions (GET, optimisé avec index)

CONSEILS POUR L'ÉQUIPE

Organisation du Travail en Équipe

- Utiliser les Issues GitHub pour tracker chaque tâche (une issue = une sous-tâche)
- Réunion quotidienne de 15 min : ce que j'ai fait, ce que je vais faire, blocages
- Chaque feature branch est revue par un autre membre avant merge
- Ne jamais pousser directement sur main ou dev

Pièges à Éviter

- Ne JAMAIS utiliser float/double pour les montants d'argent – utiliser BigDecimal (Java) ou Decimal (Python)
- Ne JAMAIS mettre de credentials en clair dans le code – utiliser des variables d'environnement
- Ne JAMAIS exposer les entités JPA directement – toujours utiliser des DTOs
- Toujours configurer des timeouts sur les appels HTTP inter-services (évite les deadlocks)
- Tester avec TestContainers et non H2 en mémoire (comportements différents)

Ressources Utiles

- Spring Boot : <https://spring.io/projects/spring-boot>
- FastAPI : <https://fastapi.tiangolo.com>
- Docker Documentation : <https://docs.docker.com>
- Kubernetes : <https://kubernetes.io/docs>
- Kong Gateway : <https://docs.konghq.com>
- OpenTelemetry : <https://opentelemetry.io/docs>

Bonne chance à toute l'équipe ! Ce projet est ambitieux mais réalisable avec de la rigueur et de la méthode.